LANGCHAIN 1.4+ • TOOL CALLING & AGENT MECHANICS

Native Tool Calling & Manual Execution Loops

Understanding Schema Binding, Model Invocations, and Multi-Turn Conversation Loops

A comprehensive deep dive into the inner workings of tool calling. Explains the `@tool` decorator, parameter schema generation, `model.bind_tools()`, and how to implement the classic two-step manual tool execution loop from scratch.

Target Notebook: `3-toolcall.ipynb`

Generation Date: November 01, 2026

Language & Runtime: Python 3.14 • Virtualenv (.venv)

Inference Engine: Groq LPU (openai/gpt-oss-20b)

B. Learning Objectives

By studying this notebook and guide, you will master:

- **The Tool Calling Contract:** Understand the division of labor between the LLM (which outputs structured JSON requests) and your application (which executes the code).
- **The `@tool` Decorator:** Turn regular Python functions into model-compatible tools using type hints and docstrings.
- **Tool Schema Binding:** Use `model.bind_tools()` to inject tool definitions into the model's system payload.
- **Parsing Tool Call Payloads:** Inspect `AIMessage.tool_calls`, extracting `name`, `args`, and unique `id`.
- **Manual Tool Execution Loops:** Build the multi-turn conversational loop manually: User $ ightarrow$ Model Tool Request $ ightarrow$ Python Tool Execution $ ightarrow$ ToolMessage $ ightarrow$ Final Model Response.
- **Conversation History Management:** Maintain sequential state in the `messages` list.

C. Prerequisites & Setup

> **Fast Inference Provider: Groq**
> This notebook leverages **Groq** for ultra-fast model inference (`openai/gpt-oss-20b`), which executes responses in ~1.00 second.

Variable / Key	Setting	Notes
`GROQ_API_KEY`	Stored in local `.env`	Required by `langchain-groq` for authentication.
Target Model	`groq:openai/gpt-oss-20b`	High-speed open-weights model hosted on Groq LPU hardware.

D. Topic Overview: What is Tool Calling?

Large Language Models cannot directly access real-time databases, browse the web, or run computations without external help. **Tool calling** (also called function calling) is the standardized protocol that bridges this gap:

1. Your application gives the LLM a list of tool descriptions (names, descriptions, parameter types).
2. When a user asks a question requiring external data, the LLM decides *which* tool to call and extracts the arguments.
3. **Crucial Principle:** The LLM does **not** execute the code. The LLM simply stops and outputs a structured request: `call get_weather with location="Delhi"`.
4. Your Python runtime executes the function locally, captures the return value, and wraps it in a `ToolMessage`.
5. Your application sends the tool result back to the LLM so it can read the observation and formulate an answer for the user.

E. Notebook Walkthrough & Cell Dissection

Cell 0: Markdown Section Header

```
### Tool Calling
```

Cell 1: Groq Initialization & Initial Invocation

```
import os
from langchain.chat_models import init_chat_model

os.environ["GROQ_API_KYE"] = os.getenv("GROQ_API_KEY")

model = init_chat_model("groq:openai/gpt-oss-20b")
response = model.invoke("Tell me something about Mbappe.")
response
```

Code & Execution Analysis:

- Notice the typo `os.environ["GROQ_API_KYE"]` in line 3. Fortunately, `os.getenv("GROQ_API_KEY")` had already populated the true environment via `.env`, so `init_chat_model` found the real key seamlessly.
- The model responded with a detailed breakdown of footballer Kylian Mbappé in 1.3 seconds, demonstrating the speed of Groq's inference engine.

Cell 2: Defining & Binding Tools

```
from langchain.tools import tool

@tool
def get_weather(location: str) -> str:
    """Get the weather at a location"""
    return f"It's sunny in {location}"

model_with_tools = model.bind_tools([get_weather])
```

Key Mechanics:

1. `@tool` **:** A decorator that converts a Python function into a `BaseTool` instance.
2. **Type Hints & Docstring:** `location: str` and the docstring `"""Get the weather at a location"""` are automatically converted into a JSON Schema definition:

   ```
   {
       "name": "get_weather",
       "description": "Get the weather at a location",
       "parameters": {"type": "object", "properties": {"location": {"type": "string"}}}
   }
   ```

3. `model.bind_tools([get_weather])` **:** Returns a new runnable model wrapper with tool definitions permanently attached to its requests.

Cell 3: Triggering a Tool Call Request

```
response = model_with_tools.invoke("What's the weather like in Delhi?")
print(response)

for tool_call in response.tool_calls:
    print(f"Tool: {tool_call['name']}")
    print(f"Args: {tool_call['args']}")
```

```
PRINTED EXECUTION OUTPUT

content='' additional_kwargs={...} tool_calls=[{'name': 'get_weather', 'args': {'location': 'Delhi'}, 'id':
'fc_623e26ba-d2e8-49ca-b7b4-69f9accf453c', 'type': 'tool_call'}]
Tool: get_weather
Args: {'location': 'Delhi'}
```

Critical Observations:

- `content=''` : The model generated zero natural language text because it chose to call a tool.
- `tool_calls` : Contains the structured payload with `name='get_weather'` and parsed arguments `{'location': 'Delhi'}` .

Cell 4: The 2-Step Manual Tool Execution Loop

```
# Tool Execution Loops

# 1. You ask, but the AI doesn't know live data
messages = [{"role": "user", "content": "What's the weather in Delhi?"}]
ai_msg = model_with_tools.invoke(messages)
messages.append(ai_msg)

# 2. Program runs the tool for the AI
for tool_call in ai_msg.tool_calls:
    tool_result = get_weather.invoke(tool_call)
    messages.append(tool_result)

# 3. Pass results back to model for final response
final_response = model_with_tools.invoke(messages)
print(final_response.text)
```

```
FINAL PRINTED RESULT

The weather in Delhi is sunny.
```

Step-by-Step Architectural Trace:

1. **Step 1:** `messages` begins with the user question. The model inspects the question and returns `ai_msg` with a tool call request.
2. **Step 2:** We append `ai_msg` to `messages` so the conversation history includes the model's intent.
3. **Step 3:** `get_weather.invoke(tool_call)` executes the Python function using the arguments generated by the model. It automatically creates a `ToolMessage` matching the tool call ID.
4. **Step 4:** We append `tool_result` to `messages` .
5. **Step 5:** We invoke `model_with_tools.invoke(messages)` again. Now the model sees the original question, its own tool call request, and the tool's actual output! It synthesizes the final conversational answer: "*The weather in Delhi is sunny.*"

Cell 5: Inspecting the Accumulated Message History

```
messages
```

```
FULL STATE INSPECTION

[
  {'role': 'user', 'content': "What's the weather in Delhi?"},
  AIMessage(content='', tool_calls=[{'name': 'get_weather', 'args': {'location': 'Delhi'}, 'id':
'fc_d4d8...'}]),
  ToolMessage(content="It's sunny in Delhi", name='get_weather', tool_call_id='fc_d4d8...')
]
```

Explanation: Demonstrates how conversational context accumulates in LangChain. Every step is transparently visible in the state list.

F. End-to-End Tool Execution Sequence Diagram

The sequence diagram below illustrates the exact two-turn handshake executed in Cell 4:

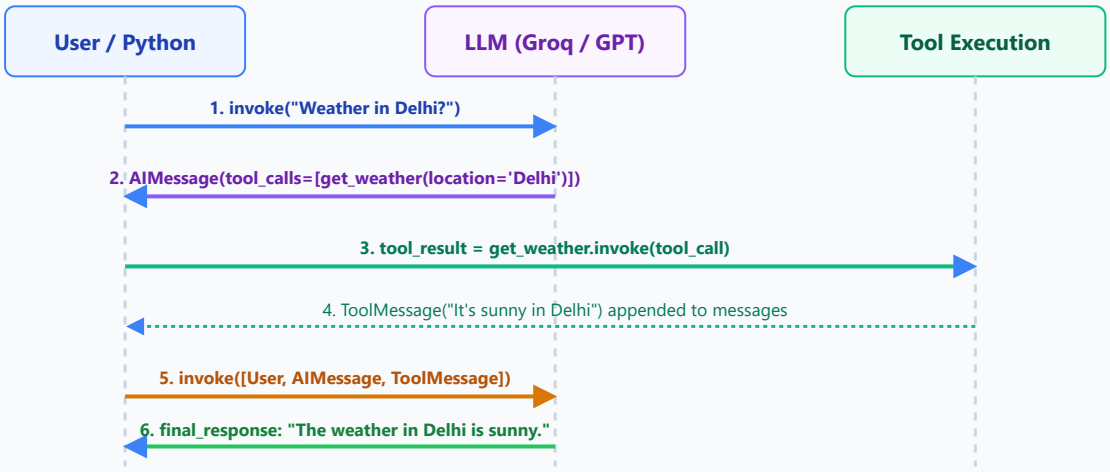


Figure 3.1: Sequence Diagram of the 2-Turn Manual Tool Calling Loop

G. Important APIs & Classes Reference

Construct	Type	Input	Output	Description
`@tool`	Decorator	Python function	`BaseTool`	Parses function annotations & docstrings into model tool schemas.
`model.bind_tools`	Method	`List[BaseTool]`	`RunnableBinding`	Binds schemas to chat model API parameters.
`response.tool_calls`	Attribute	N/A	`List[Dict]`	Structured list of tool requests emitted by the model.
`tool.invoke(tool_call)`	Method	`tool_call` dict	`ToolMessage`	Executes tool and creates linked ToolMessage with matching ID.

H. Common Errors & Debugging

Trap 1: Mismatched tool_call_id in ToolMessage

Symptom: API returns `400 Invalid Message Sequence` or model hallucinates.
Cause: If you construct a `ToolMessage` manually without matching the `id` from `tool_call['id']`, the LLM cannot link the observation to its corresponding request. Always use `tool.invoke(tool_call)` to guarantee ID synchronization.

Trap 2: Missing Return Types or Argument Types

Symptom: Tool schema has `"type": "null"` or arguments are passed as unparsed strings.
Cause: Failing to include Python type hints (e.g. `location: str`). The `@tool` decorator uses type inspection to build the JSON Schema.

I. Practical Example: Multi-Tool Dispatcher

```
tools_by_name = {"get_weather": get_weather}

for tool_call in ai_msg.tool_calls:
    tool_func = tools_by_name.get(tool_call['name'])
    if tool_func:
        result = tool_func.invoke(tool_call)
        messages.append(result)
```

Using a dictionary lookup dynamically routes multiple distinct tools without hardcoded `if/elif` statements.

J. Key Takeaways

- The model **never** runs tools directly; it generates structured JSON arguments for your host environment to execute.
- `@tool` requires both clear Python type annotations and descriptive docstrings.
- A complete tool execution loop requires two model invocations: Turn 1 produces the tool call request; Turn 2 digests the `ToolMessage` observation and answers the user.
- Automated agents (such as `create_agent` from Notebook 1) wrap this exact loop inside an automated state machine.

K. Practice Exercises & Knowledge Checks

Question 1 (Conceptual): What is the difference between model.bind_tools() and create_agent()?

Answer: `model.bind_tools()` only teaches the model about tools; you must manually write the execution loop to run the tool and send results back. `create_agent()` constructs an automated state graph (LangGraph) that handles tool execution and multi-step reasoning automatically.

Question 2 (Code Tracing): In Cell 3, why does response.content evaluate to an empty string?

> **Answer:** Because the model decided to request a tool execution. Its entire response payload was directed into `response.tool_calls`, reserving conversational text for after the tool result is received.

Question 3 (Debugging): If an environment variable is set as os.environ["GROQ_API_KYE"], why did init_chat_model("groq:...") still work?

> **Answer:** Because `load_dotenv()` had already loaded the true `GROQ_API_KEY` from the `.env` file directly into `os.environ`. The typo line merely set an extra unused variable (`GROQ_API_KYE`).

L. Further Learning & Documentation

- LangChain Tools & Tool Calling Documentation
- Groq Supported Models & Hardware Overview
- Previous study guide in sequence: **2-modelintegration.pdf** (Interaction Modes).